



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N°04 Modelado jerárquico

NOMBRE COMPLETO: Loza Regino Miguel Ángel

N° de Cuenta: 319520011

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 19/09/2026

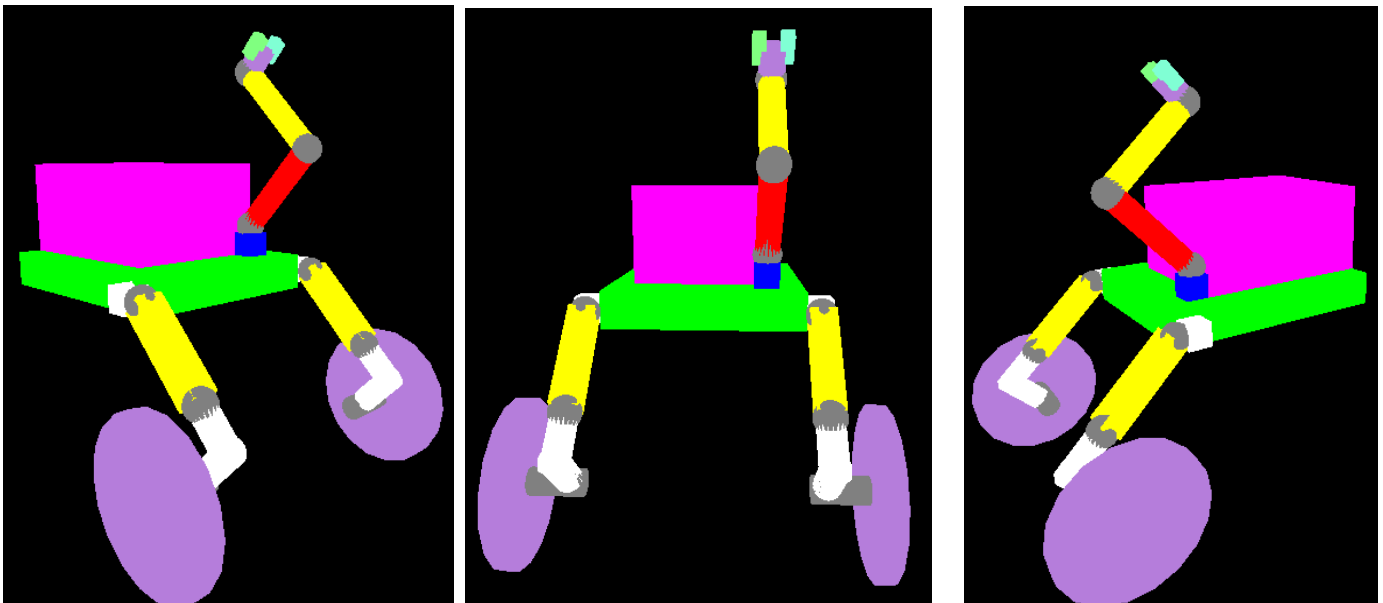
CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Terminar de construir el rover.

En concreto se debe terminar de construir

- Cabina
- Base
- Dos llantas delanteras
- Brazo



Para poder realizar el modelo del rover, se usaron diferentes estructuras.

Abordaremos su constructo explicando las siguientes partes:

Cabina, base, dos llantas delanteras, brazo.

Iniciaremos por la cabina.

En este fragmento se construye y se dibuja la cabina del rover, que además funciona como la pieza raíz de toda la jerarquía. Primero se reinicia model a la identidad (está en el origen) y lo trasladamos a $(0, 4, -4)$, que es el centro del objeto completo.

Luego lo movemos a (1, 2, 0) para llegar al centro de donde queremos que este la cabina y guardamos esa posición en modelaux **ANTES** de escalar, para que la base y los demás hijos hereden la posición, pero no el tamaño.

Después aplicamos el scale para redimensionar el cubo a la forma deseada. A decir verdad, me ha costado mucho entender el concepto de la jerarquía.

Pero hasta aquí tenemos la cabina y el modelo esta y se va a seguir construyendo desde el centro de esta.

```
//articulacion1 hasta articulación5 sólo son puntos de rotación o articulación, en este caso no dibujaremos esfera
model = glm::mat4(1.0);

model = glm::translate(model, glm::vec3(0.0f, 4.0f, -4.0f)); //NOS POSICIONAMOS EN EL CENTRO DEL OBJETO X AMARILLA
modelaux = model; //guardamos la matriz de modelo para que la base se mueva con el objeto
// Creando la cabina del rover ¿CUÁNTAS UNIDADES MEDIRÁ EN Z?
model = glm::translate(model, glm::vec3(1.0f, 2.0f, 0.0f)); //PARA LLEGAR AL CENTRO DE LA CABINA a PARTIR DEL ORIGEN
modelaux = model;
model = glm::scale(model, glm::vec3(8.0f, 4.0f, 6.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
//la línea de proyección solo se manda una vez a menos que en tiempo de ejecución se programe cambio entre proyeco
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.0f, 1.0f); // color magenta
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh(); //dibuja cubo y pirámide triangular
//meshList[3]->RenderMeshGeometry(); //dibuja las figuras geométricas cilindro, cono, pirámide base cuadrangular
//sp.render(); //dibuja esfera
```

En este bloque construimos la base, nuevamente nos posicionamos desde el origen de la cabina y colocamos el cubo redimensionado en el lugar necesario, guardamos la traslación y el escalado queda fuera.

Así mismo una de las partes mas importantes es modelaux2 ya que aquí guardaremos la posición del centro de la base, ya que de ella nos apoyaremos para poder realizar las siguientes ramas de jerarquías en las cuales modelaremos las partes faltantes.

```
model = modelaux;
//NOS POSICIONAMOS EN EL CENTRO DE LA BASE
model = glm::translate(model, glm::vec3(-1.0f, -2.75f, 0.0f));
modelaux = model; //guardamos la matriz de modelo para conectar los siguientes objetos a la base
model = glm::scale(model, glm::vec3(10.0f, 1.5f, 8.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh();

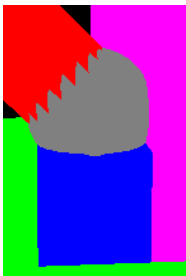
model = modelaux;
modelaux2 = model; //guardamos la matriz de modelo para conectar los siguientes objetos a la base
/*
```

Iniciamos con la construcción del brazo del rover

Partiremos del centro de la base usando model = modelaux2

Este bloque corresponde al primer cubo, que será la base del brazo de rover, el cual es un cubo rescaldo y trasladado. Así mismo se muestra la construcción de la que será la primera articulación (la esfera gris) que tiene la característica de que girara gracias a la función `articulacion1()`

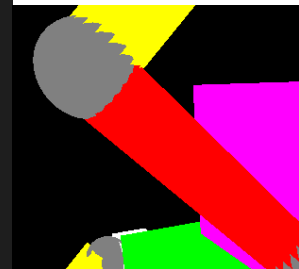
```
//=====
//Brazo del rover
//BASE PARA EL BRAZO
model = modelaux2;
model = glm::translate(model, glm::vec3(-3.5f, 1.25f, 2.5f));
//no hay scale
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.0f, 0.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMesh();
//articulación del brazo que rota con teclado F
model = glm::translate(model, glm::vec3(0.0f, 0.8f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()),
    glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();
```



Modelaremos el brazo 1 y su respectiva articulación

```
//brazo 1
model = modelaux;
model = glm::rotate(model, glm::radians(45.0f),
    glm::vec3(0.0f, 0.0f, 1.0f)); //rotar alrededor de z 45°
model = glm::translate(model, glm::vec3(0.0f, 2.3f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 4.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

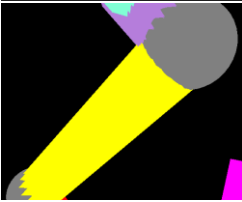
//articulación 2 rotación con G
model = modelaux;
//articulación del brazo que rota con teclado
model = glm::translate(model, glm::vec3(0.0f, 2.3f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()),
    glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();
```



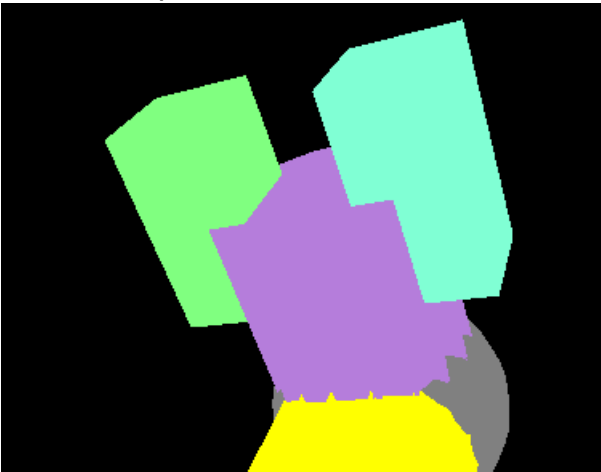
Siguiendo exactamente la misma estructura se modela el siguiente brazo y su respectiva articulación (brazo 2, articulación 3). Esta ultima articulación será la encargada de mover la pinza

```
//brazo 2
model = modelaux;
model = glm::translate(model, glm::vec3(2.3f, 0.0f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f)); //rotar alrededor de z -90°
model = glm::scale(model, glm::vec3(0.5f, 4.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

//articulación 3 rotación con H
model = modelaux;
//articulación del brazo que rota con teclado
model = glm::translate(model, glm::vec3(2.3f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()), glm::vec3(0.0f, 0.0f, 1.0f)); //La r
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();
```



Para terminar el brazo del rover, se construyó la pinza, la cual fue formada por mera estética a partir de 3 cubos escalados, a pesar de que la práctica solo pidió 1.



```

//Pinza
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.75f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(+180.0f), glm::vec3(0.0f, 0.0f, 1.0f)); //rotar alrededor de z -90°
model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.71f, 0.49f, 0.86f); //Color lila
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

//pinza 1
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.5, -0.5f));
modelaux = model;
//no necesito un rotate
model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.50f, 1.0f, 0.5f); //Color aqua
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMeshGeometry();

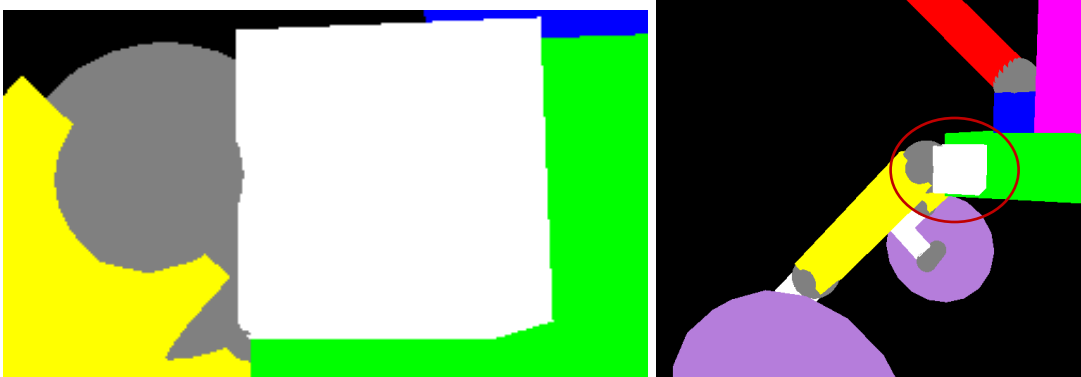
//pinza 2
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0, 1.0f));
modelaux = model;
//no necesito un rotate
model = glm::scale(model, glm::vec3(0.5f, 1.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.50f, 1.0f, 0.83f); //Color aqua
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMeshGeometry();

```

Y como parte final del ejercicio se nos solicito crear la estructura de las llantas delanteras.

Se inicio por usar modelaux2 con la abandonar la jerarquía de la rama del brazo del rover e iniciar una nueva jerarquía desde el centro de la base, tal como se solicitaba en el diagrama jerárquico.

Se inicio por crear un pequeño cubo de color blanco que separa esta estructura del cuerpo de la base y una esfera la cual en un inicio me pareció buena idea que contara con rotación, pero por la naturaleza de la estructura considero que no tendría que rotar aun así de ser necesaria se agregó como comentario la línea que le permite la rotación.



```

//=====
//Inicio del eje frontal izquierdo
//Cubo de apoyo que separa al soporte del cuerpo
model = modelaux2;
model = glm::translate(model, glm::vec3(-4.5f, 0.0f, 4.5f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f); //blanco
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMeshGeometry();

//articulación de 1 de eje frontal izq
//A mi consideración no debería rotar, ya que es un apoyo fijo
model = modelaux;
model = glm::translate(model, glm::vec3(-0.8f, 0.0f, 0.0f));
//model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

```

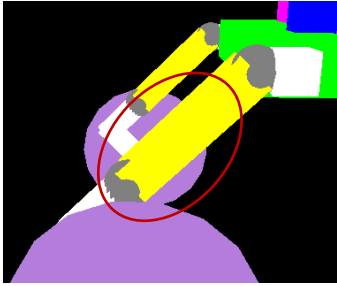
Se procedió con la construcción del soporte y la esfera correspondiente. Nuevamente a pesar de que ser esta esfera un punto marcado en el diagrama, a mi parecer por la naturaleza de la estructura no debería rotar, sin embargo, se agrega la estructura comentada necesaria para generar su rotación de ser el caso.

```

//soporte frontal izquierdo
model = modelaux;
model = glm::translate(model, glm::vec3(-1.5f, -1.4f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(4.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMeshGeometry();

//articulación de 2 de eje frontal izq
model = modelaux;
model = glm::translate(model, glm::vec3(-1.5f, -1.5f, 0.0f));
//model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f); //gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

```

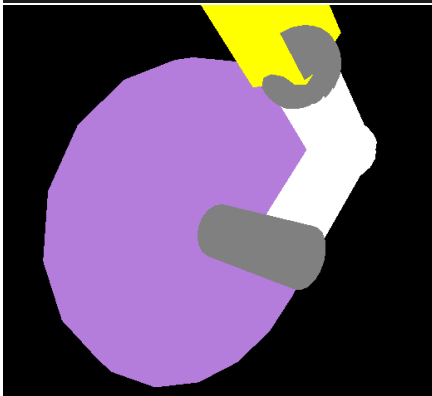


Se continua con el modelado de la pata y la parte de sujeción de la rueda del rover
Las cuales se modelaron a partir de cilindros.

```
//Parte de pata superior
model = modelaux;
model = glm::translate(model, glm::vec3(-0.75f, -0.75f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(-45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f); //blanco
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

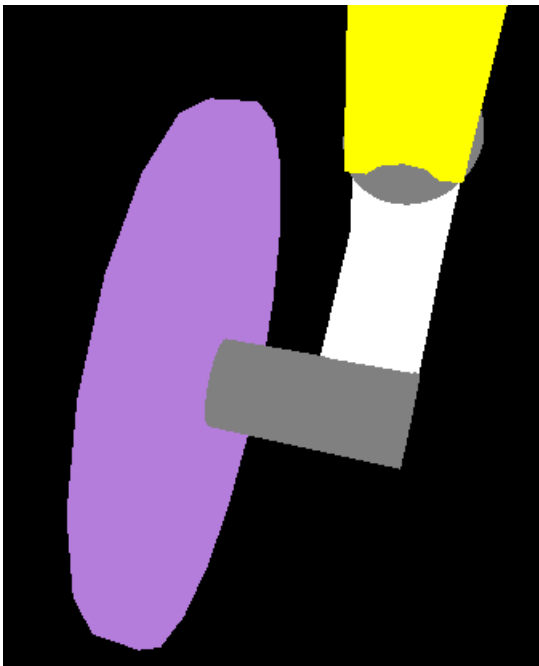
//Parte de pata Inferior
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f); //blanco
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

//Parte de pata conexión a la rueda
model = modelaux;
model = glm::translate(model, glm::vec3(1.0f, -1.0f, 0.50f));
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();
```



Y por último para concluir con una de las estructuras frontales, se modela la rueda a partir de un cilindro rotado en el eje x y re escalado limitando mucho al eje y para poder conseguir esa forma. A la rueda se le agrego la rotación con la función articulation4().

```
//Rueda
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, 1.25f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(3.0f, 0.5f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.71f, 0.49f, 0.86f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();
```



Si bien es cierto en el diagrama indicaba que para el eje frontal solo se debían usar 5 volúmenes, me pareció poco preciso es por eso que decidí implementar 3 volúmenes con la finalidad de que se viera lo más parecido al rover del diagrama.

Debo mencionar que en el reporte se omitirá la explicación de los bloques de código de la estructura de la llanta frontal derecha, ya que son exactamente los mismos códigos que los de la estructura izquierda con la única diferencia de que el en las translaciones el eje Z debe ser multiplicado por -1.

Se adjunta el bloque completo de la estructura de rueda frontal derecha.

```

///=====
//Inicio del eje frontal Derecho
//Se supone que solo bastaría cambiar el eje z
//Cubo de apoyo que separa al soporte del cuerpo
model = modelaux2;
model = glm::translate(model, glm::vec3(-4.5f, 0.0f, -4.5f));
modelaux = model;
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);//blanco
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMeshGeometry();

//articulación de 1 de eje frontal drch
//A mi consideración no debería rotar, ya que es un apoyo fijo
model = modelaux;
model = glm::translate(model, glm::vec3(-0.8f, 0.0f, 0.0f));
//model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

//soporte frontal derecho
model = modelaux;
model = glm::translate(model, glm::vec3(-1.5f, -1.4f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(4.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMeshGeometry();

```

```

//soporte frontal derecho
model = modelaux;
model = glm::translate(model, glm::vec3(-1.5f, -1.4f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(4.0f, 1.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[0]->RenderMeshGeometry();

//articulación de 2 de eje frontal drch
model = modelaux;
model = glm::translate(model, glm::vec3(-1.5f, -1.5f, 0.0f));
//model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.6f, 0.6f, 0.6f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f);//gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
sp.render();

//Parte de pata superior
model = modelaux;
model = glm::translate(model, glm::vec3(-0.75f, -0.75f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(-45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f);//blanco
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

```

```

//Parte de pata Inferior
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -1.0f, 0.0f));
modelaux = model;
model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 1.0f, 1.0f); //blanco
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

//Parte de pata conexión a la rueda
model = modelaux;
model = glm::translate(model, glm::vec3(1.0f, -1.0f, -0.50f));
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 2.0f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.5f, 0.5f, 0.5f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

//Rueda
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.25f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()), glm::vec3(0.0f, 0.0f, 1.0f)); //Se activa con J
modelaux = model;
model = glm::rotate(model, glm::radians(90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(3.0f, 0.5f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0.71f, 0.49f, 0.86f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
meshList[2]->RenderMeshGeometry();

```

De esta manera se concluye la construcción del ejercicio 04.

2. Problemas presentados: No tuve problemas con el código más bien en un inicio se me complicó entender cómo es que regresábamos al centro de la base y que es lo que hacía exactamente las estructuradas de `model = modelaux`.

Después de estudiar el código a detalle mis dudas fueron resueltas y fue muy fácil e intuitivo poder continuar con el modelado.

3. Conclusión:

- a. Los ejercicios de la clase: Una vez entendido el ejercicio, no fue complejo el realizarlo, más que nada podría llegar a ser tedioso calcular algunas distancias, pero nada que no fuera inclusive divertido. Personalmente me ayudó mucho a poder comprender como es que la jerarquía se iba a construyendo, manteniendo la posición a través de `model` y como es que `model aux` nos permitía abandonar una rama de la jerarquía para poder iniciar una nueva desde un punto en específico.
- b. Comentarios generales: Realmente la explicación fue muy clara, aunque al momento de ella tuve algunas dudas, esas mismas dudas se esclarecieron durante la realización del ejercicio mismo, fue satisfactorio para mí el poder reforzar y entender mejor el tema, el ejercicio fue un gran complemento.